

Range Minimum Queries: Analysis of Space-Time Trade-offs

Tamer Emad

Karlsruhe Institute of Technology, Germany

1 Introduction

This report presents an empirical evaluation of several Range Minimum Query (RMQ) implementations. The objective is to analyze the trade-off between the auxiliary space required for preprocessing and the time complexity of subsequent queries. The submitted work consists of a C++ benchmarking suite (`main.cpp`) with the different implementations, the generator (`input-generator`) used to create the input data for the benchmarking, this report, along with the plots, visualizing the relationship between input size n , memory consumption, and query latency.

2 Methods

The RMQ problem is defined as: For a given array $a[N]$ of size N , and a query $q : [l, r]$ (or rather a large number of them), the RMQ is finding the minimum value within range $[l, r]$, for each query q . There is a wide spectrum of approaches to this problem, from naive linear scans to advanced data structures. This report implements and compares a sample of them.

2.1 Implemented Algorithms

- **Naive Approach:** No preprocessing; computes the minimum via a linear scan for every query. Only a pointer to the array is stored.
- **Pointer-Based Segment Tree:** Is a variant of segtrees where each Node is constructed as a struct with pointers to the left and right child, and ints containing the min value and the range cover. The tree is built recursively on initiate. This implementation is easily extensible for more complex problems than RMQ.
- **Optimized Segment Tree:** Is the more optimized implementation, getting rid of pointers and other variables and only storing a single, $2N$ sized array. where each node i is a parent of $2i$, $2i + 1$. This brings much better cache utilization.
- **Sqrt Decomposition:** Stores the answers (RMQ) for $\sqrt{N}$ blocks, each of size $\sqrt{N}$. Where every query needs to sweep a maximum of $2\sqrt{N}$ plain elements, and $\sqrt{N}$ blocks.
- **Prefixed Sqrt Decomposition:** Is a Prefix-optimized variant, where additionally the prefix and **suffix** minima of each block are also stored in two arrays of size N . Which eliminates the need for plain element sweeping.
- **Multilevel Sqrt Decomposition:** Furthermore, this is an optimization where a second Sqrt Decomposition is implemented on the blocks array of the first. While also making the first array's block size smaller $\sqrt{\sqrt{N}}$. This implementation offers more speedup in query time as the maximum number of blocks swept per query is $\sqrt{|blocks_1|} = N^{3/8}$.
- **Sparse Table:** Which is (using power-of-two intervals) to store the minimum. Where any query range $[l, r]$ is solved as the min between exactly two cells in the sparse table.
- **Cartesian Trees:** Constructs a binary tree where the root is the minimum element of the array and its children are the Cartesian trees of the left and right subarrays. This transforms the RMQ problem into a Lowest Common Ancestor (LCA) problem, where the minimum of range $[l, r]$ is the LCA of the nodes corresponding to indices l and r .
- **Brute-Force Precomputation:** Pre-calculates every possible answer in an $n \times n$ matrix.

The theoretical complexities of these methods are summarized in Table 1. And the application of the complexities to this particular problem task instance.

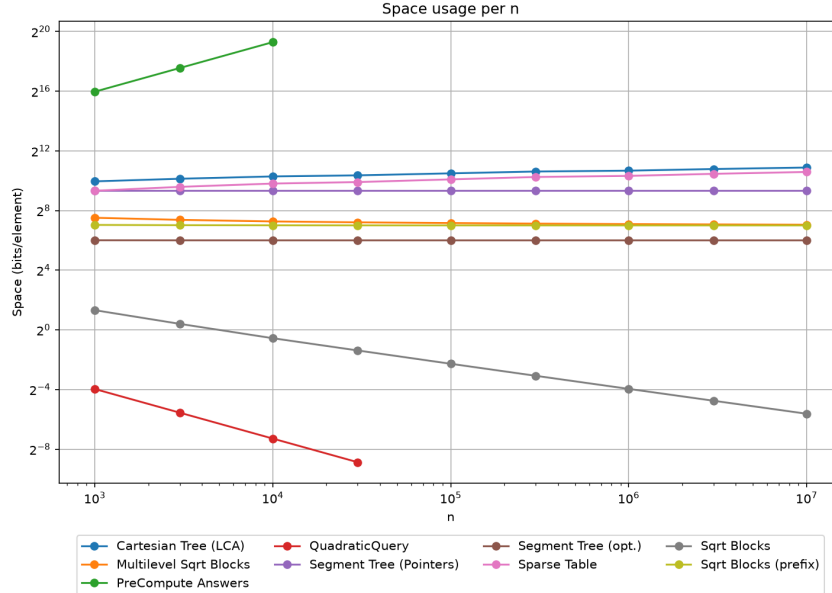
■ **Table 1** Theoretical Complexities and Total Execution Time ($n = 10^7, Q = 2 \cdot 10^6$)

Algorithm	Space	Build Time	Query Time	Total Expected Time
Naive	$O(1)$	$O(1)$	$O(n)$	≈ 8000.0 s
Segment Tree (opt.)	$O(n)$	$O(n)$	$O(\log n)$	≈ 22.4 ms
Segment Tree (Pointers)	$O(n)$	$O(n \log n)$	$O(\log n)$	≈ 110.4 ms
PreCompute Answers	$O(n^2)$	$O(n^2)$	$O(1)$	≈ 40000.0 s
Sparse Table	$O(n \log n)$	$O(n \log n)$	$O(1)$	≈ 92.8 ms
Sqrt Blocks	$O(\sqrt{n})$	$O(n)$	$O(\sqrt{n})$	≈ 2644.0 ms
Sqrt Blocks (prefix)	$O(n)$	$O(n)$	$O(\sqrt{n})$	< 2644.0 ms
Multilevel Sqrt Blocks	$O(n)$	$O(n)$	$O(n^{3/8})$	≈ 337.3 ms
Cartesian Tree (LCA)	$O(n \log n)$	$O(n \log n)$	$O(1)$	≈ 92.8 ms

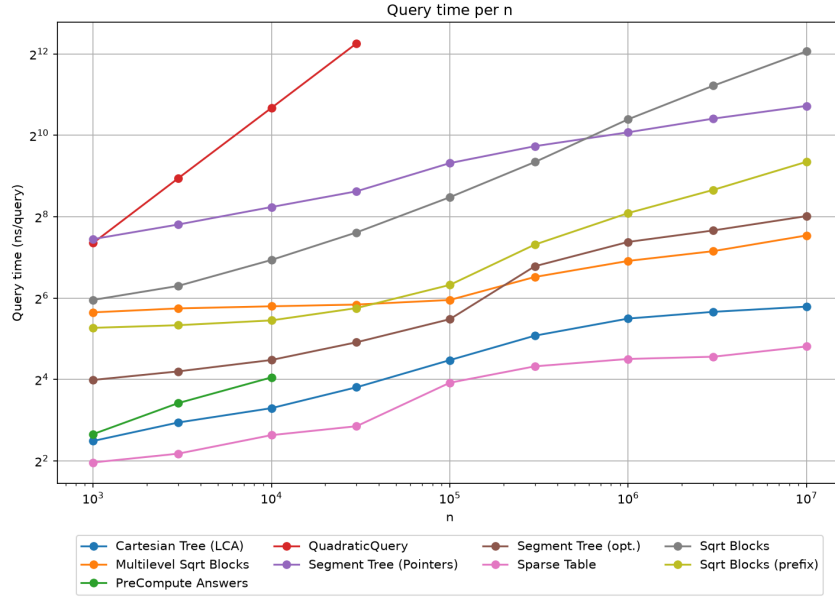
Assumption: Calculations assume 2500 operations take ≈ 1 ns (2.5 GHz). Total Time = Build Time + (Query Time $\times Q$).

According to the theory, the fastest methods are those achieving $O(1)$ query time, while the smallest is the Naive method. However, when also accounting for the build up / precompute time, the optimized segment tree is the smallest theoretically (for this exact problem constraints), as it takes $O(n)$ to build and only $Q \times \log n$ total query time.

3 Results



■ **Figure 1** Space usage per element relative to input size n .



■ **Figure 2** Query time (ns/query) as a function of input size n .

3.1 Analysis of Results

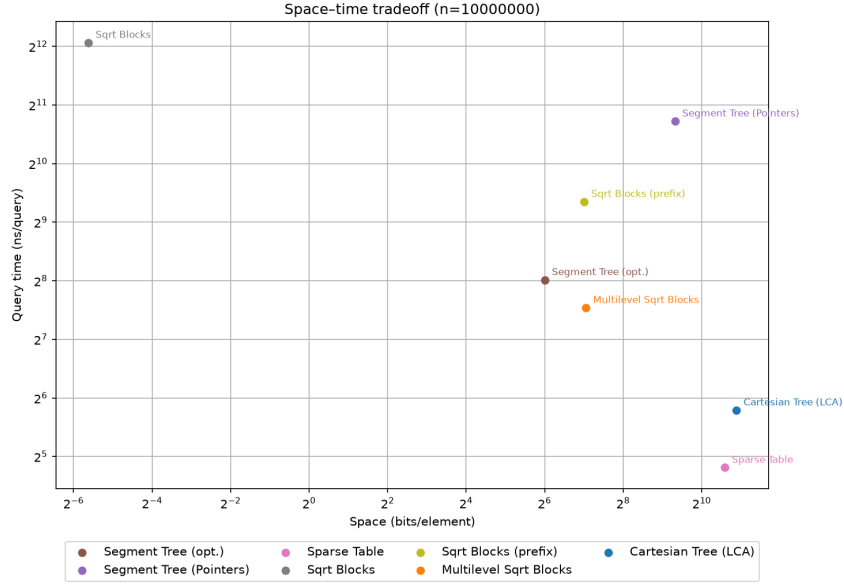
The empirical results generally align with theoretical expectations. However, there is a constant factor of 0.5 less than expected in theory and this is probably due to randomization of the input instead of worst case. as well as a combination of factors from compiler optimization and caching. In Figure 2, the Naive approach (QuadraticQuery) shows the steepest growth, while Sparse Tables and Cartesian Trees maintain a nearly flat line, confirming $O(1)$ query time. Similarly, Figure 1 confirms that the PreCompute method grows quadratically, while the Naive approach uses negligible space. Interestingly, the Multilevel Sqrt is the lowest sloped line in Figure 2 where it even crosses the optimized segment tree to perform better at $N > 10^5$ this shows that it is optimized more for this range of inputs. whereas, in the lower $N < 10^4$ it performs even worse than the prefixed Sqrt Blocks. The segment tree is expected to overtake any Sqrt Decomposition algorithm with large enough N . Unless maybe it is a fractal Sqrt Decomposition?

3.1.0.1 Space and Time Fidelity

Numerically, the space usage matches the theory closely. The Sparse Table's usage grows linearly with n (with a $\log n$ slope), while the Sqrt-based methods stay strictly within $O(n)$ limits. But there is still more variations in the constant factors which correspond directly to how the algorithm is implemented (i.e. two pre and suf arrays for Sqrt Blocks). The query times are asymptotically correct, although constants differ; for instance, the pointer-based Segment Tree is noticeably slower than the array-based one due to pointer indirection and cache misses.

3.1.0.2 Pareto Optimality

Referring to Figure 3, the Pareto-optimal algorithms, providing the best speed for a certain space budget, are the **Sparse Table** and, only a factor of 2 slower, the **Cartesian Tree** for



■ **Figure 3** The space-time trade-off for $n = 10^7$ elements.

time efficient requirements, Whereas **Sqrt Blocks** is best viable for low-memory environments. Meeting in the middle, with a more balanced trade-off at $O(N)$ memory and $O(\log N)$ time per query, is the **Segment Tree (opt.)** alongside the **Multilevel Sqrt Blocks**. At the end, **Sparse Table** is the practically optimal choice as $O(n \log n)$ space is usually acceptable for modern memory capacities in exchange for constant-time queries. It is also worthy to note that the Multilevel Sqrt drops query time by x4 while keeping the efficient space usage that is due to low space usage from the 2nd level Sqrt decomposition where the time gain is much higher. Also notable, is how the pointers Segment tree takes up 8x more space than the optimized one, as every Node stores the values, the range left and right as well as the pointers to the children. This can be further optimized but it was not pursued further in this report. Also it might be interesting to test how a trinary (or d-nary) tree might work by doing a threeway (d-way) split in each Node.

3.1.0.3 Absolute Performance and Bottlenecks

The fastest methods operate in the range of a few dozen nanoseconds per query (exactly 28.0ns). At this scale, the bottleneck is no longer the algorithmic complexity, but rather the hardware architecture. Specifically, **L1/L2 cache misses** become the primary limit.